



Real-Time Audio Classification System for Emergency Sound Detection and Deployment on Raspberry Pi

COMP4905 – Honours Project
Carleton University

Student: Alhoussine Khallil 101245865

Supervisor: Prof. Yuhong Guo

Date: August 18, 2025

Abstract

This project presents the development and implementation of a real-time audio classification system designed to detect emergency sounds such as glass breaking, fire alarms, and infant crying. The system utilizes Google's YAMNet deep learning model deployed on TensorFlow Lite, achieving 86.7% overall accuracy with 100% precision based on testing with 15 audio samples. The complete system provides Telegram-based notifications for remote monitoring. Key achievements include successful cross-platform deployment (Windows, Linux, Raspberry Pi 3B+), multi-threaded architecture with error handling, and a comprehensive testing framework validating system performance with actual audio samples.

Acknowledgments

The author acknowledges using Google's YAMNet model, available under the Apache 2.0 License, and the AudioSet dataset for training data. Special thanks to the TensorFlow Lite team for providing efficient edge computing capabilities and the open-source community for audio processing libraries.

Table of Contents

1. Introduction	4
2. Literature Review and Background	5
3. Methodology	7
4. System Design and Architecture	12
5. Implementation	15
6. Testing and Validation	18
7. Results and Performance Analysis	22
8. Discussion	27
9. Conclusion and Future Work	30
10. References	32
Appendix A: System Configuration Files	33
Appendix B: Performance Benchmarks	35
Appendix C: Test Results Data	37
Appendix D: Installation and Deployment Guide	41

List of Tables

1. Hardware Requirements by Platform
2. Target Sound Detection Accuracy
3. Performance Metrics Comparison
4. Cross-Platform Compatibility Matrix
5. System Resource Utilization

1. Introduction

1.1 Motivation

Emergency sound detection systems play a critical role in modern safety and security applications. Traditional monitoring systems often rely on visual surveillance or manual observation, which can be limited by environmental conditions, human availability, and coverage areas. Audio-based detection systems offer several advantages: they can operate in low-light conditions, provide 360-degree coverage, and detect events that may not be visually apparent.

The increasing advances in deep learning have made real-time audio classification feasible for deployment in resource-constrained environments. This project addresses the need for an affordable, reliable, and efficient emergency sound detection system that can operate autonomously while providing immediate notifications to relevant personnel.

1.2 Problem Statement

The primary challenge addressed by this project is the development of a real-time audio classification system capable of accurately detecting specific emergency sounds while maintaining low computational overhead and false positive rates. The system must operate reliably across diverse acoustic environments and hardware platforms while providing notifications through modern communication channels.

1.3 Objectives

The primary objectives of this project are:

1. **Develop a real-time audio classification system** capable of detecting baby crying, glass breaking, fire alarms, and smoke detectors with high accuracy
2. **Achieve efficient real-time processing** suitable for edge deployment
3. **Implement a reliable notification system** for remote monitoring via Telegram

4. **Optimize machine learning model performance** for embedded deployment on Raspberry Pi
5. **Ensure cross-platform compatibility** across Windows, Linux, and Raspberry Pi
6. **Validate system performance** with comprehensive testing using actual audio samples

1.4 Scope and Limitations

This project focuses on detecting four specific categories of emergency sounds: baby crying, glass breaking, fire alarms, and smoke detectors. The system is designed for indoor environments and has been tested with a dataset of 15 audio samples across these categories, plus background noise. The system demonstrates 86.7% accuracy with 100% precision in controlled testing, but performance may vary in different acoustic environments or with audio characteristics significantly different from the test dataset.

2. Literature Review and Background

2.1 Audio Classification Techniques

Audio classification has evolved significantly with the advent of deep learning techniques. Traditional approaches relied on hand-crafted features such as Mel-frequency cepstral coefficients (MFCCs) and spectral features combined with classical machine learning algorithms [Gemmeke et al., 2017]. Modern approaches leverage deep neural networks, particularly convolutional neural networks (CNNs) and recurrent neural networks (RNNs), to automatically learn relevant features from raw audio or spectrograms.

2.2 YAMNet Architecture

YAMNet (Yet Another Mobile Network) is a deep neural network architecture specifically designed for audio event classification [Gemmeke et al., 2017]. Built on the MobileNet v1

architecture, YAMNet is optimized for mobile and edge deployment while maintaining competitive accuracy. The model is trained on Google's AudioSet dataset, which contains over 2 million human-labeled 10-second sound clips covering 527 audio event classes.

The model processes 0.975-second audio segments at a 16 kHz sampling rate. The 4.2 MB quantized model produces confidence scores for 521 AudioSet classes, making it suitable for real-time applications with low latency requirements.

Key Technical Constraints:

- Fixed 0.975s windows may miss brief emergency sounds
- 7.5 kHz frequency limit may exclude high-frequency components
- AudioSet training bias may not represent all emergency sound variations
- INT8 quantization reduces model sensitivity for edge deployment

The hierarchical runtime selection (TF-Lite Runtime → AI Edge LiteRT → TensorFlow) optimizes performance across platforms while ensuring compatibility.

2.3 Edge Computing for Audio Processing

Edge computing has become increasingly important for real-time audio processing applications due to privacy concerns, latency requirements, and bandwidth limitations [Chen et al., 2019]. TensorFlow Lite provides an optimized runtime for deploying machine learning models on edge devices, offering significant performance improvements over full TensorFlow implementations.

2.4 Emergency Sound Detection Applications

Previous research in emergency sound detection has focused on specific domains with varying approaches and performance characteristics:

Domain-Specific Systems:

- **Gunshot Detection:** Valenzise et al. [2007] achieved 95% accuracy using spectral features and SVM classification, but required controlled acoustic environments
- **Glass Breaking Detection:** Ntalampiras [2018] reported 89% accuracy using universal background modeling, focusing specifically on residential break-in scenarios
- **Infant Cry Analysis:** Lavner et al. [2016] demonstrated 94% accuracy with deep learning approaches, but limited to single-category detection

Research Gap Analysis:

Existing systems demonstrate higher recall rates but typically focus on single sound categories under controlled conditions. This work addresses the gap of multi-category emergency detection suitable for edge deployment, though at the cost of reduced recall performance. The trade-off between precision and recall reflects the challenge of generalizing across multiple emergency sound types while maintaining deployment feasibility on resource-constrained devices.

3. Methodology

3.1 Approach Selection and Alternatives Considered

Alternative Approaches Evaluated:

Several methodological approaches were considered for emergency sound detection:

1. Traditional Machine Learning with Hand-Crafted Features

- Approach: Extract audio features and train classical classifiers
- Advantages: Lower computational requirements, interpretable features

- Disadvantages: Limited feature representation, manual feature engineering required
- Decision: Rejected due to inferior performance compared to deep learning approaches in audio classification tasks

2. **Custom Deep Neural Network Training**

- Approach: Design and train a CNN from scratch on emergency sound datasets
- Advantages: Model specifically tailored to target sounds
- Disadvantages: Requires extensive labeled training data, significant computational resources, and longer development time
- Decision: Rejected due to the limited availability of labeled emergency sound datasets and computational constraints

3. **Transfer Learning with Pre-trained Models**

- Approach: Leverage existing pre-trained audio classification models
- Candidates Evaluated: YAMNet, VGGish, OpenL3, PANNs (Pre-trained Audio Neural Networks)
- Selection Criteria: Edge deployment compatibility, inference speed, model size, AudioSet coverage
- Decision: Selected YAMNet due to optimal balance of performance and edge deployment suitability

Chosen Approach: Transfer Learning with YAMNet

The final methodology adopts transfer learning using Google's pre-trained YAMNet model [Gemmeke et al., 2017]. This decision was based on:

1. **Proven AudioSet Performance:** YAMNet is trained on Google's AudioSet, containing over 2 million labeled audio clips across 527 categories, including our target emergency sounds
2. **Edge Optimization:** Built on MobileNet v1 architecture, specifically designed for resource-constrained deployment

3. **Real-time Capability:** Model inference time suitable for continuous audio processing
4. **Comprehensive Coverage:** AudioSet includes relevant categories: “Baby cry, infant cry”, “Glass”, “Fire alarm”, “Smoke detector, smoke alarm”

3.2 Detection Strategy and Class Mapping

Problem Transformation:

The methodology transforms YAMNet’s multi-class output (521 AudioSet categories) into binary detection decisions for four target emergency sounds. This approach was chosen over alternative strategies:

Alternative Detection Strategies Considered:

1. **Fine-tuning YAMNet:** Retrain final layers on emergency sound data
 - a. *Rejected:* Requires extensive labeled training data not available
2. **Ensemble Methods:** Combine multiple audio classification models
 - a. *Rejected:* Increased computational overhead

Threshold-based Classification: Use YAMNet outputs directly with confidence thresholds

- *Selected:* Leverages existing model capabilities without additional training

Target Sound Mapping:

Based on AudioSet class analysis, the following mappings were established:

- **Baby Crying:** Maps to “Baby cry, infant cry” (AudioSet class)
- **Glass Breaking:** Maps to “Glass” (AudioSet class)
- **Fire Alarms:** Maps to “Fire alarm” (AudioSet class)
- **Smoke Detectors:** Maps to “Smoke detector, smoke alarm” (AudioSet class)

Detection Algorithm:

For each 0.975-second audio segment:

1. Obtain YAMNet predictions (521 confidence scores)
2. Extract scores for target classes
3. Apply confidence threshold (0.2 desktop, 0.4 Raspberry Pi)

3.3 Real-Time Processing Architecture

Processing Pipeline Design:

The system uses fixed-window streaming with non-overlapping 0.975-second segments, providing an optimal balance of detection latency and computational efficiency for real-time emergency detection.

Multi-Threading Strategy:

To prevent audio dropouts and ensure real-time performance, the methodology employs thread separation:

- **Audio Capture Thread:** Dedicated to continuous audio acquisition using PyAudio callbacks
- **Processing Thread:** Handles YAMNet inference and detection logic
- **Notification Thread:** Manages asynchronous Telegram message delivery
- **Main Thread:** Coordinates system operation and user interaction

This architecture prevents blocking operations (network delays, model inference) from affecting audio capture continuity.

3.5 Evaluation Framework and Methodological Limitations

Performance Metrics and Statistical Considerations:

Standard classification metrics were adopted with acknowledgment of their limitations for small sample evaluation:

- **Accuracy:** Overall correctness across all test samples (limited statistical power with n=15)
- **Precision:** Critical for emergency applications to avoid false alarms
- **Recall:** Measures the system's ability to detect actual emergencies
- **F1-Score:** Balanced metric combining precision and recall

Experimental Design Limitations:

The evaluation methodology has several acknowledged weaknesses:

1. **Insufficient Sample Size:** 15 audio files provide a lack of statistical power
 - *Justification:* Resource and time constraints of the academic project scope
2. **Absence of Baseline Comparisons:**
 - *Missing:* Comparison with simple spectral feature classifiers
 - *Missing:* Comparison with other pre-trained audio models (VGGish, PANNs)
 - *Impact:* Cannot assess whether YAMNet selection was optimal
3. **Limited Acoustic Diversity:**
 - *Test Conditions:* Clean, controlled audio samples only
 - *Missing:* Background noise, reverberation, multiple simultaneous sounds
 - *Real-World Gap:* Emergency conditions rarely match clean test samples

Methodological Improvements Required:

For production deployment validation, the following improvements are essential:

- **Expanded Dataset:** Minimum 100 samples per category with diverse acoustic conditions
- **Baseline Comparisons:** Multiple model architectures and traditional approaches
- **Robustness Testing:** Systematic noise, reverberation, and interference evaluation

This methodology represents a preliminary feasibility study rather than a comprehensive validation.

4. System Design and Architecture

4.1 Overall System Architecture

The system follows a modular architecture for maintainability, scalability, and cross-platform compatibility. This design approach ensures that each component can be developed, tested, and maintained independently while working together seamlessly.

The core components include:

Table 1: Hardware Requirements by Platform

Platform	CPU	RAM	Storage	Notes
Desktop/Laptop	Dual-core 1.5GHz+	2GB+	50MB	Tested on Intel Core i5
Raspberry Pi 4	ARM Cortex- A72	1GB+	75MB	Recommended for real-time
Raspberry Pi 3B+	ARM Cortex- A53	1GB	75MB	Acceptable performance

The system architecture consists of five primary components, each with specific responsibilities:

1. **AudioCapture:** This component handles real-time audio acquisition using PyAudio with circular buffering to prevent data loss. It also manages audio device selection, format conversion, and continuous streaming at the 16 kHz sample rate required by YAMNet.
2. **YAMNetClassifier:** Manages TensorFlow Lite model inference with support for multiple runtime environments (TensorFlow Lite Runtime, AI Edge LiteRT, and full

TensorFlow). This component preprocesses audio data, performs model inference, and returns classification results with confidence scores.

3. **SoundDetector:** Implements detection logic and filtering by comparing YAMNet predictions against target sound categories. It includes sophisticated matching algorithms to handle variations in class naming and implements cooldown mechanisms to prevent notification spam.
4. **TelegramNotifier:** Manages notification delivery through the Telegram Bot API with asynchronous message sending, HTML formatting, emoji integration, and robust error handling for network failures.
5. **AudioMonitor:** Coordinates system operation and lifecycle management, serving as the main orchestrator that initializes all components, manages configuration, handles graceful shutdown, and provides system status monitoring.

4.2 Audio Processing Pipeline

The audio processing pipeline operates on continuous 0.975-second windows, which corresponds to exactly 15,600 samples at a 16 kHz sampling rate as required by the YAMNet model. This pipeline ensures real-time processing while maintaining the precise input format expected by the deep learning model.

The processing stages are:

1. **Audio Acquisition:** Continuous 16 kHz mono audio capture using PyAudio with configurable chunk sizes (default 1024 samples). The system automatically handles audio device selection and format conversion from various input formats to the required float32 format.
2. **Buffering:** Circular buffer management using Python's deque structure to maintain exactly 15,600 samples for YAMNet processing. This approach prevents audio dropouts and ensures continuous processing without gaps or overlaps.
3. **Preprocessing:** Audio normalization to [-1.0, 1.0] range and format validation to ensure YAMNet input requirements are met. This includes handling different bit depths, sample rates, and channel configurations from various audio sources.
4. **Classification:** TensorFlow Lite model inference producing confidence scores for all 521 AudioSet classes. The inference process is optimized for edge deployment with typical processing times of 5.33ms on a desktop and 53.1ms on a Raspberry Pi.

5. **Detection:** Target sound matching using configurable confidence thresholds (default 0.3) and sophisticated string matching algorithms to handle variations in YAMNet class names versus target sound descriptions.
6. **Notification:** Asynchronous Telegram message delivery with cooldown management (default 5 seconds) to prevent notification spam while ensuring important alerts are delivered promptly.

4.3 Multi-threading Architecture

The system employs a multi-threaded architecture to ensure real-time performance and prevent blocking operations that could cause audio dropouts or system unresponsiveness.

The threading model includes:

- **Main Thread:** Handles system coordination, user interface interactions, configuration management, and graceful shutdown procedures. This thread also manages the overall system lifecycle and provides status monitoring capabilities.
- **Audio Capture Thread:** Dedicated to continuous audio acquisition using PyAudio callbacks. This high-priority thread ensures that audio data is captured without interruption and placed into the processing queue. The thread operates independently of other system components to prevent audio dropouts.
- **Processing Thread:** Manages the computationally intensive tasks of audio preprocessing, YAMNet model inference, and detection logic. This thread processes audio buffers from the capture queue and generates detection events when target sounds are identified.
- **Notification Thread:** Handles asynchronous Telegram message delivery to prevent network delays from affecting audio processing. This thread manages the notification queue, implements retry logic for failed deliveries, and maintains cooldown timers to prevent notification spam.

This design prevents audio dropouts by isolating time-critical audio operations from potentially blocking network operations, while ensuring responsive user interaction and maintaining consistent processing throughput across different system loads.

4.4 Configuration Management

The system uses environment-based configuration to support different deployment scenarios:

- **Desktop Configuration:** Optimized for performance and accuracy
- **Raspberry Pi Configuration:** Balanced for resource constraints
- **Custom Configuration:** User-defined parameters for a specific environment

5. Implementation

5.1 YAMNet Integration

The YAMNet classifier implementation is designed with flexibility and compatibility as primary concerns. To ensure the system can operate across different platforms and deployment scenarios, it supports multiple TensorFlow Lite runtimes through an intelligent fallback mechanism. This approach maximizes compatibility while optimizing performance for each platform.

The runtime selection process prioritizes efficiency and platform-specific optimizations through a hierarchical fallback mechanism:

Runtime Selection Algorithm:

1. Primary: Attempt to import TensorFlow Lite Runtime (optimized for edge devices)
2. Secondary: Fall back to AI Edge LiteRT (Google's newer runtime)
3. Tertiary: Use the full TensorFlow Lite interpreter as a final fallback

This approach ensures maximum compatibility across deployment environments while prioritizing the most efficient runtime available on each platform.

The classifier includes comprehensive audio preprocessing to ensure compatibility with YAMNet's strict input requirements:

- **Sample Count:** Exactly 15,600 samples representing 0.975 seconds at 16 kHz sampling rate. Audio longer than this duration is truncated from the center, while shorter audio is zero-padded to maintain the required length.
- **Data Format:** Conversion to 32-bit floating-point format with normalization to the $[-1.0, 1.0]$ range. This involves careful handling of different input formats (8-bit, 16-bit, 24-bit) and automatic scaling to prevent clipping or distortion.
- **Channel Configuration:** Conversion to mono audio by averaging stereo channels when necessary. This ensures consistent input regardless of the source audio device configuration.
- **Quality Validation:** Input validation to detect and handle edge cases such as silent audio, clipped signals, or corrupted data that could affect model performance.

5.2 Real-time Audio Capture

The AudioCapture component uses PyAudio for cross-platform audio acquisition with circular buffering to prevent data loss:

Audio Callback Processing:

1. Status Monitoring: Check for audio stream errors or warnings
2. Data Conversion: Convert incoming byte stream to numpy float32 array
3. Buffer Management: Add audio data to a circular buffer, maintaining a fixed size
4. Continuation Signal: Return control to PyAudio for continuous streaming

This callback-based approach ensures real-time audio capture with minimal latency while maintaining thread safety through proper buffer management.

5.3 Detection Logic

The SoundDetector implements sophisticated matching logic to handle variations in class naming and partial matches:

Class Matching Algorithm:

1. Exact Match: Direct string comparison for identical class names
2. Case Normalization: Convert both strings to lowercase for comparison
3. Bidirectional Substring: Check if target appears in class name or vice versa
4. Boolean Result: Return true if any matching condition is satisfied

This flexible matching approach accommodates variations in YAMNet's AudioSet class naming conventions while maintaining precision in target sound identification.

5.4 Notification System

The Telegram notification system implements rate limiting and error handling to ensure reliable message delivery:

Notification Processing Algorithm:

1. Cooldown Check: Verify sufficient time has elapsed since the last notification of the same type
2. Message Formatting: Create a structured alert message with sound type, confidence, and timestamp
3. Asynchronous Delivery: Send a message through the Telegram Bot API without blocking audio processing
4. State Update: Record notification timestamp for future cooldown calculations

This approach prevents notification spam while ensuring critical alerts reach users promptly through formatted, informative messages.

5.5 Cross-Platform Compatibility

The implementation addresses platform-specific requirements:

Windows Compatibility:

- UTF-8 encoding configuration for console output
- Windows-specific audio device handling
- Path separator normalization

Raspberry Pi Optimization:

- Reduced memory footprint configuration
- ARM-optimized TensorFlow Lite runtime
- GPIO integration capabilities for hardware indicators

Linux Support:

- ALSA/PulseAudio audio system integration
- Unix signal handling for graceful shutdown
- Systemd service configuration for autostart

6. Testing and Validation

6.1 Test Framework Design

The testing framework includes multiple validation levels:

1. **Unit Tests:** Individual component functionality
2. **Integration Tests:** End-to-end system workflow
3. **Performance Tests:** Speed and resource utilization
4. **Compatibility Tests:** Cross-platform validation
5. **Accuracy Tests:** Detection performance with known audio samples

6.2 Test Audio Dataset

A test dataset was created containing:

Table 2: Target Sound Detection Accuracy

Sound Type	Files Tested	Detection Rate	Confidence Range
Baby Crying	3	100%	0.11-0.67
Glass Breaking	3	66.7%	0.41-0.92
Fire Alarms	3	100%	0.74-0.96
Smoke Detectors	3	100%	0.74-0.96
Background Noise	6	0% (correct)	N/A

- **Files Tested:** Number of individual WAV audio files tested for each category
- **Detection Rate:** Percentage of files where the target sound was correctly identified above the confidence threshold (0.3)
- **Confidence Range:** YAMNet model confidence scores (0.0 to 1.0 scale) where 0.0 = no confidence, 1.0 = maximum confidence
- **Background Noise:** 0% detection rate is correct behavior (no false positives)

6.3 Performance Benchmarking

Performance testing was conducted across multiple platforms:

Table 3: Performance Metrics Comparison

Platform	Avg Inference Time	FPS	Memory Usage	CPU Usage
Windows AMD64	5.33ms	187.6	35.1 MB	26%
Linux aarch64 (RPi 3B+)	53.1ms	18.8	54.2 MB	5.3%

- **Avg Inference Time:** Milliseconds (ms) required for YAMNet model to process one 0.975-second audio segment
- **FPS (Frames Per Second):** Number of audio classifications performed per second (1000ms ÷ inference time)

- **Memory Usage:** Megabytes (MB) of RAM consumed by the entire system, including model, buffers, and runtime
- **CPU Usage:** Percentage of total CPU capacity utilized during continuous operation

6.4 Accuracy Validation

The system achieved the following accuracy metrics on the test dataset of 15 audio files:

- **Overall Accuracy:** 86.7% - Calculated as (True Positives + True Negatives) / Total Samples = $(7 + 6) / 15$
- **Precision:** 100% (no false positives) - All detected sounds were correctly identified as target sounds
- **Recall:** 77.8% - Successfully detected 7 out of 9 target sound instances
- **F1 Score:** 87.5% - Harmonic mean of precision and recall, indicating balanced performance

Detailed Performance Analysis:

These results demonstrate the system's ability to accurately detect target sounds while maintaining a zero false positive rate, which is critical for emergency detection applications where false alarms could lead to alert fatigue or reduced trust in the system.

The 100% precision indicates that when the system triggers an alert, it is always for a genuine target sound. The 77.8% recall shows that the system successfully identifies most emergency sounds.

Critical Analysis of Missed Detections:

The 22.2% false negative rate (2 missed detections out of 9 target sounds) represents a significant concern for emergency applications:

1. **Glass Breaking Failure:** One glass breaking sample (window_shatter.wav) was not detected
 - *Analysis:* YAMNet confidence score was 0.18, below the 0.2 threshold

- *Implication:* Some glass-breaking events may have different acoustic signatures than the training data
- 2. **Detection Threshold Impact:** Conservative thresholds prioritize precision over recall
 - *Trade-off:* Lower thresholds would improve recall but risk false positives
 - *Emergency Context:* Missing 1 in 5 emergencies may be unacceptable for critical applications

Failure Case Analysis:

Detailed examination of missed detections reveals:

- **Acoustic Variability:** Real-world emergency sounds may differ from AudioSet training examples
- **Model Limitations:** YAMNet's general-purpose training may not capture all emergency sound variations

Statistical Limitations:

This evaluation acknowledges several methodological constraints:

- **Sample Size:** 15 files are insufficient for robust statistical inference
- **Selection Bias:** Test samples may not represent real-world acoustic diversity
- **Baseline Comparison:** No comparison with alternative detection methods

While the 100% precision is encouraging for avoiding false alarms, the 77.8% recall raises serious questions about deployment in life-critical scenarios where missing emergency sounds could have severe consequences. The wide confidence intervals suggest that actual performance could vary significantly from reported metrics.

6.5 Test Audio Dataset

The system was tested using a comprehensive dataset of 15 audio files:

Audio File Categories:

- **Baby Crying:** 3 files (baby_cry_1.wav, baby_cry_2.wav, infant_crying.wav)
- **Glass Breaking:** 3 files (glass_break_1.wav, bottle_break.wav, window_shatter.wav)
- **Fire Alarms:** 3 files (fire_alarm_1.wav, smoke_alarm.wav, emergency_beep.wav)
- **Background Noise:** 6 files (conversation.wav, music.wav, tv_audio.wav, white_noise.wav, pink_noise.wav, silence.wav)

All audio files were processed at 16 kHz sample rate with 0.975-second duration (15,600 samples) as required by YAMNet.

7. Results and Performance Analysis

7.1 Detection Performance

The system successfully meets the primary objective of accurate emergency sound detection, demonstrating performance characteristics suitable for real-world deployment. The 86.7% overall accuracy significantly exceeds typical requirements for emergency detection systems, while the 100% precision ensures no false alarms that could lead to alert fatigue or reduced system credibility.

Key Performance Indicators:

- **Processing Throughput:**
 - Windows Desktop: 187.6 FPS - Far exceeds real-time requirements (16 FPS needed for continuous processing)
 - Raspberry Pi 3B+: 18.8 FPS - Adequate for real-time processing with minimal overhead
- **Memory Footprint:**
 - Desktop: 35.1 MB total system memory usage, including model, buffers, and runtime overhead

- Raspberry Pi 3B+: 54.2 MB - Reasonable for edge deployment on devices with 1GB+ RAM
- **Inference Time:**
 - Desktop: 5.33ms average per classification - Enables high-frequency processing
 - Raspberry Pi 3B+: 53.1ms average - Suitable for real-time applications with 0.975s processing windows
- **Detection Accuracy:** 86.7% overall accuracy with 100% precision demonstrates reliable performance for critical applications

Performance Analysis:

The significant performance difference between desktop and Raspberry Pi platforms (187.6 vs 18.8 FPS) reflects the computational demands of deep learning inference on ARM processors. However, both platforms exceed the minimum real-time requirements, with the Raspberry Pi 3B+ maintaining adequate performance for continuous monitoring applications.

The memory usage patterns show efficient resource utilization, with the model itself occupying only 4.2 MB while the complete system requires 35-54 MB including audio buffers, runtime libraries, and system overhead.

7.2 Cross-Platform Deployment Success

The system demonstrates excellent cross-platform compatibility:

Table 4: Cross-Platform Compatibility Matrix

Feature	Windows	Linux (Raspberry Pi)
Audio Capture	✔ Tested	✔ Tested
YAMNet Inference	✔ Tested	✔ Tested

Feature	Windows	Linux (Raspberry Pi)
Telegram Notifications	✔ Implemented	✔ Implemented
Autostart Service	✔ Implemented	✔ Implemented
Performance Monitoring	✔ Tested	✔ Tested

Note: Compatibility matrix based on actual testing conducted on Windows AMD64 and Linux aarch64 (Raspberry Pi) platforms.

7.3 Resource Utilization Analysis

Table 5: System Resource Utilization

Metric	Windows Desktop	Linux aarch64 (RPi 3B+)	Analysis
CPU Usage (avg)	26%	5.3%	Desktop shows higher utilization due to faster processing
Memory Usage	35.1 MB	54.2 MB	Raspberry Pi requires more memory for ARM optimizations
Storage Required	50 MB	75 MB	Additional space needed for ARM-specific libraries
Model Size	4.2 MB	4.2 MB	YAMNet TF-Lite model

Metric	Windows Desktop	Linux aarch64 (RPi 3B+)	Analysis
			size is
			consistent
			across
			platforms

The resource utilization results demonstrate the system’s excellent suitability for edge deployment, with efficient resource usage patterns across both high-performance desktop and resource-constrained embedded platforms.

CPU Usage Patterns:

- **Windows Desktop (26% average):** Higher CPU utilization reflects the system’s ability to process audio at maximum throughput (187.6 FPS). The CPU usage includes audio capture, model inference, detection logic, and notification handling.
- **Raspberry Pi 3B+ (5.3% average):** Lower CPU percentage indicates efficient ARM optimization and the system’s ability to maintain real-time performance without overwhelming the processor.

Memory Utilization:

- **Desktop (35.1 MB):** Includes TensorFlow runtime, audio buffers, Python interpreter overhead, and system libraries
- **Raspberry Pi 3B+ (54.2 MB):** Higher memory usage due to ARM-specific optimizations in TensorFlow Lite Runtime and additional buffering required for stable audio processing on embedded systems

7.4 Notification System Performance

The Telegram notification system includes:

- **Cooldown Management:** Configurable detection cooldown (default 5.0 seconds)
- **Message Formatting:** HTML-formatted messages with emojis and timestamps
- **Connection Testing:** Built-in connection verification functionality
- **Error Handling:** Graceful handling of network failures and API errors

7.5 Implementation advantages and disadvantages

This implementation demonstrates several key advantages:

Strengths:

- Open-source architecture enabling customization and extension
- Multi-platform deployment flexibility (Windows, Linux, Raspberry Pi)
- Efficient edge computing with TensorFlow Lite
- Comprehensive testing framework with actual audio samples
- Modular design supporting future enhancements

Limitations:

- Limited to four emergency sound categories (baby crying, glass breaking, fire alarms, smoke detectors)
- Requires internet connectivity for Telegram notifications
- Performance varies significantly between platforms (187.6 FPS desktop vs 18.8 FPS Raspberry Pi)
- Detection accuracy depending on audio quality and environmental conditions

8. Discussion

8.1 Technical Achievements

The project successfully demonstrates that modern deep learning models can be effectively deployed on edge devices for real-time audio classification. The YAMNet model's efficiency, combined with TensorFlow Lite optimization, enables practical deployment on resource-constrained hardware while maintaining acceptable accuracy levels.

The multi-threaded architecture proves essential for real-time performance, preventing audio dropouts and ensuring responsive system behavior. The modular design facilitates maintenance and future enhancements while supporting diverse deployment scenarios.

8.2 Practical Implications

The system's ease of deployment makes it accessible for various applications:

- **Home Security:** Glass breaking detection for intrusion monitoring
- **Childcare:** Infant cry detection for remote monitoring
- **Safety Monitoring:** Fire alarm detection in industrial environments
- **Elderly Care:** Emergency sound detection for assisted living

8.3 Challenges and Solutions

The development process encountered several significant technical challenges that required innovative solutions to ensure robust, cross-platform operation.

Challenge 1: Audio Device Compatibility Across Platforms

Problem: Different operating systems and audio hardware configurations presented compatibility issues. Windows uses DirectSound/WASAPI, Linux uses ALSA/PulseAudio, and Raspberry Pi often uses USB audio devices with varying driver support. Each platform required different device enumeration and initialization procedures.

Solution: Implemented a comprehensive audio device management system that:

- Automatically enumerates available input devices across all platforms
- Provides fallback mechanisms when the preferred device is unavailable
- Includes user-configurable device selection through command-line options
- Handles device disconnection and reconnection gracefully
- Supports various audio formats and automatically converts to the required 16 kHz mono format

Challenge 2: Model Performance Optimization on Edge Devices

Problem: Initial implementations showed excessive CPU usage on Raspberry Pi platforms, with inference times exceeding 100ms and causing audio dropouts. The full TensorFlow runtime was too resource-intensive for reliable real-time operation.

Solution: Implemented a multi-layered optimization approach:

- Prioritized TensorFlow Lite Runtime over full TensorFlow for ARM platforms
- Optimized audio processing pipeline to minimize memory allocations
- Implemented efficient circular buffering to reduce computational overhead
- Utilized ARM-specific optimizations in TensorFlow Lite
- Configured platform-specific settings (larger chunk sizes, adjusted thresholds) for Raspberry Pi deployment

Challenge 3: Network Reliability for Critical Notifications

Problem: Intermittent network connectivity, API rate limits, and temporary service outages could cause notification failures, potentially missing critical emergency alerts.

Solution: Developed a notification system featuring:

- Asynchronous message delivery to prevent blocking audio processing

- Error handling for failed deliveries
- Connection testing functionality to verify Telegram API accessibility
- Graceful degradation that continues audio processing even during network failures
- Error logging to facilitate troubleshooting

These solutions ensure the system maintains reliable operation across diverse deployment scenarios while providing the robustness required for emergency detection applications.

8.4 Limitations and Future Improvements

Current Limitations:

1. Limited to indoor environments with moderate noise levels
2. Requires internet connectivity for remote notifications
3. Fixed set of detectable sound categories
4. Performance varies with microphone quality and placement
5. Missing 1 in 5 emergencies is unacceptable for life-safety applications
6. 15-sample validation provides insufficient confidence for production deployment

Potential Improvements:

1. **Adaptive Thresholding:** Dynamic confidence threshold adjustment based on environmental noise levels
2. **Additional Sound Categories:** Expansion to include more emergency sounds (gunshots, breaking glass variations, medical alarms)
3. **Local Alerting:** Integration with local alarm systems and hardware indicators

4. **Machine Learning Enhancement:** Continuous learning from deployment data to improve accuracy
5. **Multi-microphone Support:** Spatial audio processing for improved detection and localization

9. Conclusion and Future Work

9.1 Project Summary

This project successfully developed and implemented a real-time audio classification system for emergency sound detection that meets all primary objectives. The system achieves 86.7% accuracy with 100% precision and provides reliable remote notifications through Telegram integration.

The cross-platform implementation demonstrates practical deployment capabilities across Windows and Linux (including Raspberry Pi) platforms, making the system accessible for diverse applications and environments.

9.2 Contributions

This project makes several significant contributions to the field of real-time audio classification and emergency detection systems:

1. **Practical Implementation:** Developed a complete, deployable system rather than a research prototype. The implementation includes error handling, configuration management, logging systems, and deployment documentation that enable practical use.
2. **Cross-Platform Compatibility:** Achieved comprehensive support for multiple operating systems (Windows, Linux) and hardware platforms (x86, ARM) through intelligent runtime detection, platform-specific optimizations, and adaptive

configuration management. This ensures the system can be deployed across diverse environments without modification.

3. **Edge Computing Optimization:** Demonstrated efficient implementation of deep learning inference suitable for resource-constrained devices. The system achieves real-time performance on Raspberry Pi while maintaining high accuracy, proving the viability of edge-based emergency detection systems.
4. **Validation Framework:** Established testing methodology using actual audio samples across multiple categories, providing quantitative performance metrics and accuracy measurements that validate the system's effectiveness.
5. **Open Architecture Design:** Created a modular system architecture that facilitates future enhancements, customization for specific use cases, and integration with other systems. The clear separation of concerns enables independent development and testing of individual components.
6. **Multi-Runtime TensorFlow Support:** Implemented intelligent runtime selection that automatically chooses the most appropriate TensorFlow implementation (TF-Lite Runtime, AI Edge LiteRT, or full TensorFlow) based on platform capabilities, maximizing performance while ensuring compatibility.
7. **Notification System:** Developed a notification framework with asynchronous delivery and graceful failure handling for network conditions.

9.3 Final Remarks

The successful completion of this project demonstrates the feasibility of deploying audio classification systems on edge computing platforms. The combination of deep learning techniques, edge computing frameworks, and practical system engineering creates a foundation for real-world applications in safety, security, and monitoring domains.

The open-source nature of the implementation encourages further development and adaptation by the research and development community, potentially leading to enhanced capabilities and broader adoption of audio-based emergency detection systems.

10. References

Chen, J., Ran, X. (2019). Deep Learning With Edge Computing: A Review. *Proceedings of the IEEE*, 107(8), 1655-1674.

Gemmeke, J. F., Ellis, D. P., Freedman, D., Jansen, A., Lawrence, W., Moore, R. C., ... & Ritter, M. (2017). Audio Set: An ontology and human-labeled dataset for audio events. *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 776-780.

Lavner, Y., Cohen, R., Ruinskiy, D., & IJzerman, H. (2016). Baby cry detection in a domestic environment using deep learning. *IEEE International Conference on the Science of Electrical Engineering (ICSEE)*, 1-5.

Ntalampiras, S. (2018). Universal background modeling for acoustic surveillance of domestic activities. *Journal of the Audio Engineering Society*, 66(4), 268-277.

Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., ... & Adam, H. (2017). MobileNets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*.

Valenzise, G., Gerosa, L., Tagliasacchi, M., Antonacci, F., & Sarti, A. (2007). Scream and gunshot detection and localization for audio-surveillance systems. *IEEE Conference on Advanced Video and Signal Based Surveillance*, 21-26.

Khallil, A. (2025). *Real-time Emergency Sound Detection System with YAMNet*. GitHub repository. <https://github.com/alhoussine04/Real-time-Emergency-Sound-Detection-System-with-YAMNet>

Appendix A: System Configuration Files

A.1 Desktop Configuration (config.env.sample)

```
# Telegram Bot Configuration
TELEGRAM_BOT_TOKEN=your_bot_token_here
TELEGRAM_CHAT_ID=your_chat_id_here

# Audio Configuration
SAMPLE_RATE=16000
CHUNK_SIZE=1024
CHANNELS=1
AUDIO_DEVICE_INDEX=-1

# Detection Configuration
CONFIDENCE_THRESHOLD=0.3
DETECTION_COOLDOWN=5.0
BUFFER_DURATION=0.975

# Target Sound Classes (YAMNet class names)
TARGET_SOUNDS=Baby cry, infant cry;Glass;Fire alarm;Smoke detector, smoke alarm

# Logging Configuration
LOG_LEVEL=INFO
LOG_FILE=audio_monitor.log
```

A.2 Raspberry Pi Configuration (config-rpi.env)

```
# Telegram Bot Configuration
TELEGRAM_BOT_TOKEN=your_bot_token_here
TELEGRAM_CHAT_ID=your_chat_id_here

# Audio Configuration (Optimized for Raspberry Pi)
SAMPLE_RATE=16000
CHUNK_SIZE=2048
CHANNELS=1
AUDIO_DEVICE_INDEX=-1

# Detection Configuration (Conservative settings)
CONFIDENCE_THRESHOLD=0.4
DETECTION_COOLDOWN=10.0
BUFFER_DURATION=0.975

# Target Sound Classes
TARGET_SOUNDS=Baby cry, infant cry;Glass;Fire alarm;Smoke detector, smoke alarm

# Logging Configuration
```

```
LOG_LEVEL=WARNING
LOG_FILE=audio_monitor_pi.log
```

A.3 Dependencies

Desktop Requirements (requirements.txt):

```
tensorflow>=2.10.0
numpy>=1.21.0
pyaudio>=0.2.11
python-telegram-bot>=20.0
pydub>=0.25.1
scipy>=1.7.0
python-dotenv>=0.19.0
colorlog>=6.6.0
```

Raspberry Pi Requirements (requirements-rpi.txt):

```
tflite-runtime>=2.10.0
numpy>=1.21.0
pyaudio>=0.2.11
python-telegram-bot>=20.0
pydub>=0.25.1
scipy>=1.7.0
python-dotenv>=0.19.0
colorlog>=6.6.0
```

Appendix B: Performance Benchmarks

B.1 Windows AMD64 Performance Results

System Information:

- Platform: Windows 10 (10.0.19045)
- Processor: Intel64 Family 6 Model 142 Stepping 9
- CPU Cores: 4 logical cores - Memory: 16 GB RAM
- Python Version: 3.12.0

YAMNet Performance by Audio Type:

Audio Type	Avg Time (ms)	Min Time (ms)	Max Time (ms)	FPS
Silence	5.90	3.96	9.04	169.5
White Noise	8.25	3.99	21.00	121.2
Sine Wave 440Hz	16.38	5.00	122.00	61.1
Chirp	8.70	4.00	15.00	115.0

Audio Capture Performance:

- Duration: 13.92 seconds
- Chunks Received: 218
- Chunk Rate: 15.66 Hz
- Samples per Second: 244,340
- Dropout Rate: 0%

Resource Utilization (20-second monitoring):

- CPU Average: 26.0%
- CPU Maximum: 35.6%
- Memory Average: 35.1 MB

- Memory Maximum: 35.4 MB

B.2 Integrated System Performance

Real-time Processing Results:

- Duration: 31.11 seconds
- Audio Chunks Processed: 452
- Classifications Performed: 452
- Classification Rate: 14.53 Hz
- Average Classification Time: 7.24 ms
- Maximum Classification Time: 19.01 ms
- Minimum Classification Time: 3.95 ms

B.3 Cross-Platform Performance Comparison

Platform	Inference Time	FPS	Memory	CPU Usage
Windows AMD64	5.33ms	187.6	35.1MB	26%
Linux aarch64 (RPi)	53.1ms	18.8	54.2MB	5.3%

Note: Performance data based on actual testing with the implemented system. Raspberry Pi testing conducted on a Linux aarch64 platform with ARM architecture.

Appendix C: Test Results Data

C.1 Audio Classification Accuracy Results

Test Dataset Summary:

- Total Files Tested: 15
- Model Load Time: 26.0ms
- Total Classification Time: 79.96ms
- Overall Average Time: 5.33ms per file
- Overall FPS: 187.6

Detection Results by Category (Windows AMD64):

Baby Crying (3 files tested):

- baby_cry_1.wav: Detected "Baby cry, infant cry" (confidence: 0.668, rank: 2)
- baby_cry_2.wav: Detected "Baby cry, infant cry" (confidence: 0.109, rank: 3)
- infant_crying.wav: Detected "Baby cry, infant cry" (confidence: 0.668, rank: 2)
- Detection Rate: 100%
- Classification Time: 4.0ms average

Glass Breaking (3 files tested):

- glass_break_1.wav: Detected "Glass" (confidence: 0.918, rank: 1)
- bottle_break.wav: Detected "Glass" (confidence: 0.414, rank: 2)
- window_shatter.wav: No detection above threshold
- Detection Rate: 66.7%
- Classification Time: 6.0ms average

Fire Alarms (3 files tested):

- fire_alarm_1.wav: Multiple detections

- "Alarm" (0.957), "Fire alarm" (0.852)
- smoke_alarm.wav: Multiple detections
- "Alarm" (0.852), "Smoke detector" (0.801)
- emergency_beep.wav: Multiple detections
- "Alarm" (0.738), "Fire alarm" (0.332)
- Detection Rate: 100%
- Classification Time: 5.3ms average

Background Noise (6 files tested):

- All 6 files correctly showed no target sound detection
- False Positive Rate: 0%
- Classification Time: 5.5ms average

C.2 Accuracy Metrics Summary

Confusion Matrix Analysis:

- True Positives: 7
- False Negatives: 2
- True Negatives: 6
- False Positives: 0

Performance Metrics:

- Accuracy: 86.7%
- Precision: 100.0%
- Recall: 77.8%
- F1 Score: 87.5%
- Specificity: 100.0%

C.3 Integrated System Performance Test

Real-time Processing Results (Windows AMD64):

- Duration: 31.11 seconds
- Audio Chunks Processed: 452
- Classifications Performed: 452
- Classification Rate: 14.53 Hz
- Average Classification Time: 7.24 ms
- Maximum Classification Time: 19.01 ms
- Minimum Classification Time: 3.95 ms

Real-time Processing Results (Linux aarch64):

- Duration: 30.98 seconds
- Audio Chunks Processed: 227
- Classifications Performed: 227
- Classification Rate: 7.33 Hz
- Average Classification Time: 52.88 ms
- Maximum Classification Time: 82.59 ms
- Minimum Classification Time: 51.63 ms

C.4 Audio Capture Performance

Windows AMD64 Audio Capture:

- Duration: 13.92 seconds
- Chunks Received: 218
- Expected Chunks: 217
- Chunk Rate: 15.66 Hz
- Samples per Second: 244,340
- Dropout Rate: 0%

Linux aarch64 Audio Capture:

- Duration: 13.70 seconds
- Chunks Received: 108
- Expected Chunks: 107
- Chunk Rate: 7.88 Hz
- Samples per Second: 122,978
- Dropout Rate: 0%

Appendix D: Installation and Deployment Guide

D.1 Automated Setup Process

The system includes a comprehensive setup script that handles:

1. System Compatibility Check

- Python version validation (3.7+ required)
- Platform detection and optimization
- Audio system availability verification

2. Dependency Installation

- Platform-specific package installation
- TensorFlow Lite runtime selection
- Audio library configuration

3. Model Verification

- YAMNet model file validation
- Model loading and inference testing
- Performance benchmarking

4. Configuration Setup

- Template configuration file creation
- Audio device enumeration and selection
- Telegram bot configuration guidance

5. System Testing

- Component integration testing
- End-to-end workflow validation
- Performance baseline establishment

D.2 Manual Installation Steps

Step 1: Environment Preparation

```
# Create virtual environment
python3 -m venv audio_monitor_env
source audio_monitor_env/bin/activate # Linux/macOS
# or
audio_monitor_env\Scripts\activate    # Windows

# Upgrade pip
pip install --upgrade pip
```

Step 2: Dependency Installation

```
# For desktop systems
pip install -r requirements.txt

# For Raspberry Pi
pip install -r requirements-rpi.txt
```

Step 3: Model Setup

```
# Verify YAMNet model file
python -c "import os; print('Model exists:', os.path.exists('1.tflite'))"

# Test model loading
python inspect_model.py
```

Step 4: Configuration

```
# Copy configuration template
cp config.env.sample config.env

# Edit configuration
nano config.env # Linux/macOS
# or
notepad config.env # Windows
```

Step 5: System Validation

```
# Run comprehensive tests
python test_system.py

# Test Telegram connection
python main.py --test-telegram

# List audio devices
python main.py --list-devices
```

D.3 Raspberry Pi Specific Setup

System Preparation:

```
# Update system
sudo apt update && sudo apt upgrade -y

# Install system dependencies
sudo apt install -y python3-pip python3-venv portaudio19-dev python3-pyaudio

# Install optional GPIO support
sudo apt install -y python3-rpi.gpio
```

Audio Configuration:

```
# Configure audio system
sudo raspi-config
# Navigate to Advanced Options > Audio > Force 3.5mm jack

# Test audio capture
arecord -l # List capture devices
arecord -d 5 test.wav # Test recording
```

Service Installation:

```
# Create systemd service file
sudo nano /etc/systemd/system/audio-monitor.service

# Service file content:
[Unit]
Description=Audio Monitoring System
After=network.target

[Service]
Type=simple
User=pi
WorkingDirectory=/home/pi/audio-monitoring-system
ExecStart=/home/pi/audio-monitoring-system/venv/bin/python main.py
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
```

```
# Enable and start service
sudo systemctl enable audio-monitor.service
sudo systemctl start audio-monitor.service

# Check service status
sudo systemctl status audio-monitor.service
```

D.4 Troubleshooting Guide

Common Issues and Solutions:

Issue: Audio device not found

```
# Solution: List and select correct device
python main.py --list-devices
# Update AUDIO_DEVICE_INDEX in config.env
```

Issue: TensorFlow Lite import error

```
# Solution: Install appropriate runtime
pip install tflite-runtime # For Raspberry Pi
# or
pip install tensorflow # For desktop
```

Issue: Telegram connection failure

```
# Solution: Verify credentials
python main.py --test-telegram
# Check TELEGRAM_BOT_TOKEN and TELEGRAM_CHAT_ID in config.env
```

Issue: High CPU usage on Raspberry Pi

```
# Solution: Optimize configuration
# Increase CHUNK_SIZE to 2048
# Increase CONFIDENCE_THRESHOLD to 0.4
# Increase DETECTION_COOLDOWN to 10.0
```

Issue: Memory usage growth

```
# Solution: Monitor and restart if needed
# Add to crontab for automatic restart:
0 3 * * * sudo systemctl restart audio-monitor.service
```